

Homework #5

Name(s): Raj Shah (ras637), (), ()

Acknowledgment. We discussed the problems only with our group members and used only the resources allowed by the course policy. Any external help, if used, has been properly acknowledged.

Problem 1

A graph $G = (V, E)$ is dense if $|E| = \Theta(|V|^2)$. Using binary heaps, Prim's algorithm runs in $O(E \log V)$. Using Fibonacci heaps, the runtime improves to $O(E + V \log V)$, which becomes linear $O(E)$ for dense graphs. Describe an alternative algorithm that uses a simple data structure and runs in linear time $O(E)$ for dense graphs.

Solution.

We use **Prim's algorithm with an adjacency matrix** instead of a heap.

Since the graph is dense, we have

$$|E| = \Theta(|V|^2).$$

So an $O(V^2)$ algorithm is the same as $O(E)$ time on dense graphs.

Pseudo code

Algorithm: Prim's Algorithm with an Adjacency Matrix

1. Store the graph in an adjacency matrix $W[1..n][1..n]$, where $W[u][v]$ is the edge weight if edge (u, v) exists, and ∞ otherwise.
2. Choose any start vertex r .
3. For each vertex v , maintain:
 - **key** $[v]$: minimum weight of an edge connecting v to the current MST,
 - **parent** $[v]$: the chosen parent of v in the MST,
 - **inMST** $[v]$: whether v is already in the MST.

4. Initialize:

$$\text{key}[r] = 0, \quad \text{key}[v] = \infty \text{ for all } v \neq r.$$

5. Repeat n times:

- (a) Find a vertex u not yet in the MST with minimum **key** $[u]$.
- (b) Set **inMST** $[u] = \text{true}$.
- (c) For every vertex $v = 1, 2, \dots, n$:
 - if $v \notin \text{MST}$ and $W[u][v] < \text{key}[v]$, then update

$$\text{key}[v] = W[u][v], \quad \text{parent}[v] = u.$$

6. Output the edges (**parent** $[v], v$) for all $v \neq r$.

Proof of correctness.

This is still exactly Prim's algorithm. The only difference is the data structure used to find the minimum key vertex and to scan all incident edges. Prim's algorithm is correct because at each step it chooses the minimum-weight edge crossing the cut between the current tree and the remaining vertices. By the cut property of minimum spanning trees, this choice is always safe.

Therefore, the algorithm outputs a minimum spanning tree.

Running time.

- We perform n iterations.
- In each iteration:
 - finding the minimum key vertex takes $O(n)$,
 - scanning the full row of the adjacency matrix also takes $O(n)$.

So the total running time is

$$O(n^2).$$

For a dense graph, $|E| = \Theta(n^2)$, so

$$O(n^2) = O(E).$$

Final Answer (boxed)

Use Prim's algorithm with an adjacency matrix and no heap. It runs in $O(V^2)$ time, which is $O(E)$ when the graph is dense because $E = \Theta(V^2)$.

Problem 2

Demonstrate failure of the following algorithm on a directed graph G with edge weights in $\{-1, 0, 1\}$. The algorithm adds 1 to every edge weight, runs Dijkstra's algorithm on the new graph G' , and then outputs the corresponding path weight back in the original graph G .

Solution.

We now give a counterexample where this method returns the wrong answer.

Graph G

Let

$$V = \{s, a, b, t\}.$$

Directed edges and original weights:

$s \rightarrow a$ with weight -1 ,

$a \rightarrow b$ with weight -1 ,

$b \rightarrow t$ with weight 1 ,

$s \rightarrow t$ with weight 0 .

So there are two s - t paths:

- Direct path:

$$s \rightarrow t, \quad \text{weight } 0.$$

- Three-edge path:

$$s \rightarrow a \rightarrow b \rightarrow t, \quad \text{weight } (-1) + (-1) + 1 = -1.$$

Thus, in the original graph G , the true shortest distance from s to t is

$$\boxed{-1}.$$

Transformation to G'

Now add 1 to every edge weight:

$$w'(x, y) = w(x, y) + 1.$$

So the transformed weights are:

$$s \rightarrow a : 0, \quad a \rightarrow b : 0, \quad b \rightarrow t : 2, \quad s \rightarrow t : 1.$$

Run Dijkstra on G'

We now trace Dijkstra's algorithm from source s .

Initialization:

$$\text{dist}'(s) = 0, \quad \text{dist}'(a) = \text{dist}'(b) = \text{dist}'(t) = \infty.$$

Priority queue:

$$[(s, 0)].$$

Step 1: Extract s

$$\text{dist}'(s) = 0.$$

Relax outgoing edges from s :

$$\text{dist}'(a) = 0, \quad \text{dist}'(t) = 1.$$

Priority queue becomes:

$$[(a, 0), (t, 1)].$$

Step 2: Extract a

$$\text{dist}'(a) = 0.$$

Relax edge $a \rightarrow b$:

$$\text{dist}'(b) = 0.$$

Priority queue becomes:

$$[(b, 0), (t, 1)].$$

Step 3: Extract b

$$\text{dist}'(b) = 0.$$

Relax edge $b \rightarrow t$:

$$\text{dist}'(t) = \min(1, 0 + 2) = 1.$$

So no improvement occurs. Priority queue becomes:

$$[(t, 1)].$$

Step 4: Extract t Dijkstra returns the path

$$s \rightarrow t$$

with transformed distance

$$1.$$

What answer does the algorithm output?

The algorithm outputs the weight of that chosen path in the original graph G . The chosen path is still

$$s \rightarrow t,$$

whose original weight is

$$\boxed{0}.$$

Why this is wrong

But the true shortest path in the original graph is

$$s \rightarrow a \rightarrow b \rightarrow t$$

with total weight

$$(-1) + (-1) + 1 = \boxed{-1}.$$

So the algorithm returns

$$\boxed{0}$$

even though the correct answer is

$$\boxed{-1}.$$

Reason for failure:

Adding 1 to every edge does *not* preserve shortest paths, because paths with different numbers of edges are changed by different amounts.

If a path has k edges, then its total weight increases by exactly k . So longer paths get penalized more heavily than shorter paths, even if they were originally better. Therefore the order of path weights can change, and the shortest path in G' may not be the shortest path in G .

Final Answer (boxed)

A correct counterexample is:

$$s \rightarrow a(-1), \quad a \rightarrow b(-1), \quad b \rightarrow t(1), \quad s \rightarrow t(0).$$

In G , the shortest s - t path has weight -1 , but after adding 1 to every edge, Dijkstra on G' chooses the direct edge $s \rightarrow t$, which corresponds to original weight 0. So the algorithm fails.

Problem 3

You are given a weighted undirected graph $G = (V, E)$ with integer weights $w_e \in \{1, 2, \dots, W\}$ on each edge, where $W \leq 25$. Design and analyze an algorithm to find the minimum-weight s - t path in $O(V + E)$ time.

Solution.

Since the weights are positive integers bounded by a constant $W \leq 25$, we can transform the weighted graph into an unweighted graph of only constant-factor larger size and then run BFS.

Main idea

Replace each edge of weight w by a path of length w .

Because $W \leq 25$ is a constant, each original edge creates at most 25 edges in the new graph. So the new graph still has size $O(V + E)$.

Construction

Suppose an edge (u, v) in G has weight w .

- If $w = 1$, keep the edge as it is.
- If $w > 1$, replace it by a chain of w unweighted edges:

$$u \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_{w-1} \rightarrow v,$$

where $x_1, \dots, x_{w-1}$ are new dummy vertices.

Since the original graph is undirected, do the same in the undirected sense: each weighted edge becomes an undirected path of length w .

Call the new graph G' . Every edge in G' has unit weight.

Algorithm

Pseudo code

Algorithm

1. Construct the unweighted graph G' by replacing every edge of weight w by a path of length w .
2. Run BFS from s in G' .
3. Output the BFS distance from s to t .

Proof of correctness

Every path in the original graph G corresponds to a path in G' whose number of edges is exactly the same as the total weight of the original path.

More precisely, if a path in G uses edges of weights

$$w_1, w_2, \dots, w_k,$$

then the corresponding path in G' has length

$$w_1 + w_2 + \cdots + w_k.$$

Similarly, every path from s to t in G' corresponds to a path in G with the same total weight, after contracting each gadget back into its original weighted edge.

Thus:

$$\text{shortest weighted path in } G = \text{shortest unweighted path in } G'.$$

BFS correctly finds shortest paths in unweighted graphs, so BFS on G' returns the minimum-weight path value from s to t in G .

Running time

Let $n = |V|$ and $m = |E|$.

Each original edge of weight at most W is replaced by at most W unit edges and at most $W - 1$ new vertices. Since $W \leq 25$ is a constant:

$$|V(G')| = O(V + E), \quad |E(G')| = O(E).$$

Running BFS on G' takes

$$O(|V(G')| + |E(G')|) = O(V + E).$$

Final Answer (boxed)

Replace every edge of weight w by a path of length w , creating an unweighted graph of size $O(V + E)$ because $W \leq 25$ is constant. Then run BFS from s to t . This gives the minimum-weight path in $O(V + E)$ time.

Problem 4(a)

Suppose you have an algorithm **EdgeWeightedSP**(G, s) that solves the edge-weighted shortest path problem in $O(|E(G)|)$. Write pseudocode for **VertexWeightedSP**(H, s) that solves the vertex-weighted shortest path problem in $O(|E(H)|)$.

Solution.

We reduce vertex weights to edge weights by splitting each vertex into two copies.

Construction

Given a vertex-weighted graph $H = (V, E)$ with vertex cost $c(v)$ for each vertex v :

- For each vertex $v \in V$, create two vertices:

$$v_{\text{in}}, v_{\text{out}}.$$

- Add a directed edge

$$(v_{\text{in}}, v_{\text{out}})$$

with weight $c(v)$.

- For every original edge $(u, v) \in E(H)$, add an edge

$$(u_{\text{out}}, v_{\text{in}})$$

with weight 0.

If H is undirected, then for each undirected edge $\{u, v\}$ add both directed edges

$$u_{\text{out}} \rightarrow v_{\text{in}} \quad \text{and} \quad v_{\text{out}} \rightarrow u_{\text{in}}$$

with weight 0.

Pseudocode

Pseudo code

VertexWeightedSP(H, s)

1. Create a new edge-weighted graph G' .
2. For each vertex $v \in V(H)$:
 - create v_{in} and v_{out} in G' ,
 - add edge $(v_{\text{in}}, v_{\text{out}})$ of weight $c(v)$.
3. For each edge $(u, v) \in E(H)$:
 - add edge $(u_{\text{out}}, v_{\text{in}})$ of weight 0.
 - If H is undirected, also add edge $(v_{\text{out}}, u_{\text{in}})$ of weight 0.
4. Run **EdgeWeightedSP**(G', s_{in}).
5. For each original vertex v , output

$$\text{dist}_H(s, v) = \text{dist}_{G'}(s_{\text{in}}, v_{\text{out}}).$$

Proof of correctness

Any path in H :

$$s = v_0, v_1, v_2, \dots, v_k = v$$

corresponds in G' to:

$$s_{\text{in}} \rightarrow s_{\text{out}} \rightarrow v_{1,\text{in}} \rightarrow v_{1,\text{out}} \rightarrow \dots \rightarrow v_{\text{in}} \rightarrow v_{\text{out}}.$$

The only edges with nonzero weight are the internal edges

$$(v_{\text{in}}, v_{\text{out}})$$

which contribute exactly the vertex cost $c(v)$. Therefore the total weight of the path in G' equals the sum of vertex weights on the corresponding path in H .

So shortest paths in G' correspond exactly to shortest vertex-weighted paths in H .

Running time

The transformed graph has:

$$2|V(H)| \text{ vertices}$$

and

$$|V(H)| + O(|E(H)|)$$

edges, which is $O(|E(H)|)$ size overall.

Thus running the blackbox algorithm takes

$$O(|E(H)|).$$

Final Answer (boxed)

Split every vertex v into v_{in} and v_{out} , connect them by an edge of weight $c(v)$, and give every original graph edge weight 0 between the split vertices. Then run the edge-weighted shortest path algorithm on the transformed graph.

Problem 4(b)

Suppose you have an algorithm $\text{VertexWeightedSP}(H, s)$ that solves the vertex-weighted shortest path problem in $O(|E(H)|)$. Write pseudocode for $\text{EdgeWeightedSP}(G, s)$ that solves the edge-weighted shortest path problem in $O(|E(G)|)$.

Solution.

We now reduce edge weights to vertex weights.

Construction

Given edge-weighted graph $G = (V, E)$ with edge weight $w(u, v)$:

- Keep each original vertex $v \in V$ and assign it vertex weight 0.
- For every edge $(u, v) \in E$, create a new intermediate vertex x_{uv} with weight

$$c(x_{uv}) = w(u, v).$$

- Replace edge (u, v) by two edges:

$$u \rightarrow x_{uv}, \quad x_{uv} \rightarrow v.$$

If the original graph is undirected, connect in both directions accordingly.

Pseudocode

Pseudo code

EdgeWeightedSP(G, s)

1. Create a new vertex-weighted graph H .
2. For each original vertex $v \in V(G)$:
 - add v to H with vertex weight 0.
3. For each edge $(u, v) \in E(G)$ with weight $w(u, v)$:
 - create a new vertex x_{uv} with weight $w(u, v)$,
 - add edges (u, x_{uv}) and (x_{uv}, v) .
4. Run $\text{VertexWeightedSP}(H, s)$.
5. For each original vertex $v \in V(G)$, return

$$\text{dist}_G(s, v) = \text{dist}_H(s, v).$$

Proof of correctness

Each original edge (u, v) of weight $w(u, v)$ is replaced by a two-step path

$$u \rightarrow x_{uv} \rightarrow v,$$

where the only nonzero cost on that segment is the vertex weight of x_{uv} , namely $w(u, v)$.

Since original vertices have weight 0, the total vertex-weighted cost of any path in H is exactly the total edge-weighted cost of the corresponding path in G .

Therefore shortest paths are preserved exactly under this transformation.

Running time

We add one new vertex per edge and two new edges per edge, so

$$|V(H)| = |V(G)| + |E(G)|, \quad |E(H)| = O(|E(G)|).$$

Hence the blackbox runs in

$$O(|E(H)|) = O(|E(G)|).$$

Final Answer (boxed)

Replace each edge (u, v) of weight $w(u, v)$ by a new intermediate vertex of weight $w(u, v)$ and edges $u \rightarrow x_{uv} \rightarrow v$. Set original vertex weights to 0. Then run the vertex-weighted shortest path algorithm on the transformed graph.

Problem 5

Given a directed graph $G = (V, E)$ with positive integer edge weights, let s be the source vertex. Assume all shortest path distances satisfy $\text{dist}(s, v) \leq B$. Design a data structure D such that running Dijkstra's algorithm using D gives runtime $O(|E| + B)$.

Solution.

Because all edge weights are positive integers and all final shortest path distances are at most B , we can use a **bucket-based priority queue**. This is essentially Dial's idea.

Data structure

We maintain:

- An array of buckets

`Bucket[0..B]`,

where `Bucket[i]` stores all vertices currently having tentative distance exactly i .

- An array `dist[v]` storing the current key of vertex v .
- A boolean array `inQ[v]`.
- A pointer `current` to the smallest bucket index not yet fully processed.

Each bucket is implemented as a doubly linked list, and for each vertex we store a pointer to its node in the list so that deletion is $O(1)$.

Operations

Pseudo code

Insert(v, p)

1. Set `dist[v] = p`.
2. Insert v into `Bucket[p]`.
3. Set `inQ[v] = true`.

Time: $O(1)$.

Pseudo code

Extract-Min()

1. While `current ≤ B` and `Bucket[current]` is empty, increment `current`.
2. If `current > B`, return NULL.
3. Remove and return any vertex v from `Bucket[current]`.
4. Set `inQ[v] = false`.

Time: $O(1)$ amortized, and total scanning over all buckets is $O(B)$.

Pseudo code

Decrease-Key(v , newP)

1. Remove v from its old bucket `Bucket[dist[v]]`.
2. Set `dist[v] = newP`.
3. Insert v into `Bucket[newP]`.
4. If `newP < current`, set `current = newP`.

Time: $O(1)$.

Why Dijkstra works with this structure

In Dijkstra's algorithm with positive integer weights, when a vertex gets tentative distance d , its key is an integer. Since every true shortest path distance is at most B , we never need a bucket beyond B .

The data structure always extracts a vertex of minimum current tentative distance, exactly as Dijkstra requires.

Running time

- Each edge relaxation is processed once, contributing $O(E)$ total.
- Each **Insert** or **Decrease-Key** takes $O(1)$.
- Each vertex is extracted once.
- The pointer **current** only moves forward from 0 to B , so all bucket scanning across the whole algorithm takes $O(B)$ total.

Thus the total runtime is

$$O(E + B).$$

Final Answer (boxed)

Use an array of buckets indexed by distance values $0, 1, \dots, B$. Insert and decrease-key take $O(1)$, and extract-min takes $O(1)$ amortized with only $O(B)$ total scanning over empty buckets. Therefore Dijkstra runs in $O(E + B)$ time.

Problem 6

Describe and analyze a modification of Bellman–Ford that:

- returns a reachable negative cycle if one exists,
- sets $dist(v) = -\infty$ for every vertex reachable from that cycle,
- otherwise returns the shortest-path tree and all shortest-path distances,

all in $O(VE)$ time.

Solution.

We modify Bellman–Ford in three stages.

Step 1: Standard Bellman–Ford relaxations

Initialize:

$$dist(s) = 0, \quad dist(v) = \infty \text{ for } v \neq s.$$

Also maintain a parent pointer `parent[v]` for path reconstruction.

Run the usual Bellman–Ford relaxation for $|V| - 1$ rounds over all edges.

If after these rounds no edge can still be relaxed, then there is no reachable negative cycle, and the computed distances are correct.

Step 2: Detect a reachable negative cycle

Do one more full pass over all edges.

If there exists an edge (u, v) such that

$$dist(u) + w(u, v) < dist(v),$$

then a negative cycle is reachable from s .

Let x be such a vertex v whose distance can still be improved.

Step 3: Extract the cycle and mark all vertices reachable from it

To find an actual cycle:

1. Start at x and follow parent pointers backward for $|V|$ steps.
2. The resulting vertex must lie on a negative cycle; call it y .
3. Starting from y , keep following parent pointers until y repeats. The visited vertices form the cycle.

Now let C be the set of vertices on this negative cycle.

To find all vertices whose shortest path value should be $-\infty$, run DFS or BFS from all vertices in C in the original directed graph. Every reachable vertex can be made arbitrarily cheap by looping around the negative cycle enough times and then continuing to that vertex. So set

$$dist(v) = -\infty$$

for every such reachable vertex.

Pseudocode sketch

Pseudo code

Modified-Bellman-Ford(G, s)

1. Initialize distances and parent pointers.
2. Repeat $|V| - 1$ times:
 - relax every edge.
3. Let $x = \text{NULL}$.
4. For each edge (u, v) :
 - if $\text{dist}(u) + w(u, v) < \text{dist}(v)$, set $x = v$.
5. If $x = \text{NULL}$:
 - return the shortest-path tree and all distances.
6. Else:
 - (a) follow parent pointers from x exactly $|V|$ times to get a vertex y on a cycle,
 - (b) walk parent pointers starting at y until the walk returns to y to recover the cycle,
 - (c) run DFS/BFS from all cycle vertices,
 - (d) set $\text{dist}(v) = -\infty$ for every reached vertex,
 - (e) return the cycle and the distance array.

Proof of correctness

Case 1: No reachable negative cycle.

Bellman-Ford is correct after $|V| - 1$ rounds when no reachable negative cycle exists. So the returned distances and parent pointers define the shortest-path tree.

Case 2: A reachable negative cycle exists.

If an edge can still be relaxed on the $|V|$ th pass, Bellman-Ford guarantees that some reachable negative cycle exists. Following parent pointers for $|V|$ steps ensures we enter a cycle, since there are only $|V|$ vertices total. The cycle we recover must be a negative cycle reachable from s .

Any vertex reachable from that negative cycle has shortest-path value $-\infty$, because we can loop around the cycle arbitrarily many times, lowering the path cost without bound, and then continue to that vertex. So setting those distances to $-\infty$ is correct.

Running time

- $|V| - 1$ rounds of edge relaxation: $O(VE)$.
- One additional pass to detect improvement: $O(E)$.
- Following parent pointers and extracting the cycle: $O(V)$.
- DFS/BFS from the cycle vertices: $O(V + E)$.

Hence the overall running time is

$$O(VE).$$

Final Answer (boxed)

Run Bellman–Ford normally for $|V| - 1$ rounds, then do one extra pass. If no edge improves, return the usual shortest-path tree. If some edge improves, use parent pointers to recover a reachable negative cycle, then run DFS/BFS from that cycle and set all reachable vertices to distance $-\infty$. The total running time is $O(VE)$.

Problem 7

Let $G = (V, E)$ be a directed graph with weighted edges, where weights may be positive, negative, or zero. The vertices are partitioned into k disjoint subsets $V_1, V_2, \dots, V_k$. For each i, j , define

$$\delta(i, j) = \min\{\text{dist}(u, v) \mid u \in V_i, v \in V_j\}.$$

Describe an algorithm to compute all $\delta(i, j)$ in

$$O(VE + kV \log V)$$

time.

Solution.

The main idea is to first remove negative weights using Johnson's reweighting trick, and then run one multi-source Dijkstra per subset.

Step 1: Reweight the graph using Bellman–Ford

Add a new super-source q and connect it to every vertex $v \in V$ by an edge of weight 0.

Run Bellman–Ford from q to compute a potential function $h(v) = \text{dist}(q, v)$.

Then define reweighted edge costs:

$$w'(u, v) = w(u, v) + h(u) - h(v).$$

By Johnson's theorem, all $w'(u, v) \geq 0$, and for every pair of vertices x, y :

$$\text{dist}'(x, y) = \text{dist}(x, y) + h(x) - h(y).$$

This first step takes

$$O(VE).$$

Step 2: For each subset, run one multi-source Dijkstra

Fix a subset V_i .

We want, for every vertex v , the value

$$\min_{u \in V_i} \text{dist}(u, v).$$

Using the reweighted graph, we have

$$\text{dist}(u, v) = \text{dist}'(u, v) - h(u) + h(v).$$

So if we initialize a Dijkstra run with each source vertex $u \in V_i$ having initial key

$$-h(u),$$

then the computed value at a vertex v becomes

$$d_i(v) = \min_{u \in V_i} (-h(u) + \text{dist}'(u, v)).$$

Therefore

$$\min_{u \in V_i} \text{dist}(u, v) = d_i(v) + h(v).$$

So for each subset V_i :

1. Run a multi-source Dijkstra on the nonnegative reweighted graph.
2. Initialize every vertex $u \in V_i$ with priority $-h(u)$.
3. Let the resulting values be $d_i(v)$.
4. Recover the true minimum distance from V_i to v by

$$D_i(v) = d_i(v) + h(v).$$

Now for each subset V_j , compute

$$\delta(i, j) = \min_{v \in V_j} D_i(v).$$

Why this is correct

After reweighting:

$$\text{dist}'(u, v) = \text{dist}(u, v) + h(u) - h(v).$$

Rearranging gives:

$$\text{dist}(u, v) = \text{dist}'(u, v) - h(u) + h(v).$$

Thus for fixed i :

$$\min_{u \in V_i} \text{dist}(u, v) = h(v) + \min_{u \in V_i} (\text{dist}'(u, v) - h(u)).$$

This is exactly what the multi-source Dijkstra computes when each source $u \in V_i$ starts with initial key $-h(u)$.

Then taking the minimum over all $v \in V_j$ gives exactly

$$\delta(i, j) = \min_{u \in V_i, v \in V_j} \text{dist}(u, v).$$

So the algorithm correctly computes every entry of the matrix δ .

Running time

- Bellman–Ford once from q : $O(VE)$.
- For each subset V_i , one multi-source Dijkstra:

$$O(E + V \log V).$$

- Doing this for all k subsets gives:

$$O(kE + kV \log V).$$

Since $k \leq V$, we have

$$kE \leq VE.$$

So the total time is

$$O(VE + kV \log V).$$

Final Answer (boxed)

Use Johnson's reweighting: run Bellman-Ford once from a super-source to get potentials $h(v)$, then reweight all edges so they are nonnegative. For each subset V_i , run one multi-source Dijkstra with initial key $-h(u)$ for every $u \in V_i$. This computes $\min_{u \in V_i} \text{dist}(u, v)$ for every vertex v , and then taking a minimum over each subset V_j gives $\delta(i, j)$. The total running time is $O(VE + kV \log V)$.